DataSci 112: Principles of Data Science

Stanford University, Summer 2026

Before You Do Anything Else!!!

This is the Instructor's version of this notebook. To complete some assignments below you will need to modify its contents. Any changes that you make cannot be saved and will be lost when you close the browser.

If you want to save your work, **you need to save a copy of this notebook** to your personal Google Drive. To do this, go to File > Save a copy in Drive.

In fact, this should be the first thing you do whenever you open up a notebook that someone shares with you.

Introduction to Notebooks and Colab

In CS 106A (or another Intro to Computing course that used Python), you wrote Python code in an IDE such as Pycharm. You would first write a Python program in a file like `myscript.py`, then you would execute the program by typing `python3 myscript.py` at a terminal.

In this class, we write Python code in **notebooks**, like this one. Notebooks can run on your computer or in the cloud. **Colab** is a free Google service that offers cloud notebooks that are linked to your Google Drive.

Python Notebooks consist of several types of *cells*, of which two: **text cells**, and **code cells** are the most important.

Text Cells allow the notebook author to include any textual description as part of the notebook. The contents of these cells can contain formatting instructions that are **parsed** (processed by the software that runs the notebooks for you) and affect how the text is presented to the user.

This cell is a text cell.

How's it going?

This is also a text cell. All text in this cell is rendered without the use of any formatting

instructions.

This text cell contains some formatting instructions. They can be intruduced using the notation known as the **Markdown** markup language (pun intended). Markdown supports several types of text formatting, including **boldface**, *italics*, ~~strikethrough~~, `computer command` style, web page lines (see above),

- lists
- of
- items,

1. both, itemized (above)
2. and enumerated
3. with sublists

embedding images



> creating blockquotes
>
> > often embedded into one another

and creating code blocks:

```
for i in range(10):
   print(i)
```

There are

also

several

different levels of headings

Seeing Markdown

when you double-click text cell, in Google Colab you will see the raw Markdown formatting of the cell on the left, and the final look of the cell on the right. In Jupyter Labs (for those of you who installed Anaconda) you will simply see the raw Markdown syntax.

LaTeX

Both Google Colab and Jupyter Labs can parse and render another markup language, $\LaTeX$, which allows for rendering of mathematical formulae and such as what you see below

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

or

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & \sqrt{7} \end{pmatrix}$$

(in this class, you won't be required to learn $\LaTeX$, however we will be using it to render formulas in the notebooks shared with you whenever the need arises)

Task 1. Create a text cell below. Use a header, and some Markdown formatting (you can use the toolbar to help you insert the formatting into the text cell) to specify your name, your *Stanford email address*, and a couple of sentences about you (where you go to school, what you are studying in college or a planning to study), and why you are taking this course.

Hello! my name is **Maxim**. My email is *txdva@stanford.edu*, which was just a randomly chosen collection of characters. I go to *Brighton College*.

I'm planning to study mainly **Computer Science**, and I'm taking this course to learn

more about how to work with data.

Code cells. Code cells are for, well, writing code. The notebooks we use in DATASCI 112 are set up to process specifically code written in the Python programming language. Some examples of code cells are below.

Generally speaking, code cells contain comments and executable statements.

In [1]:
```
## this is a code cell
## it contains Python code that can be run.
## Code cells can include standard Python comments
## and in general, you should take advantage of this
## and comment your code

## Code cells can also contain Python code that, when the cell is run
## (press the button in the top left corner, or hit <Ctrl>+<Enter>)
## will get executed.

## here is some simple Python code

a = 1
print("Value of a:", a)
b = a+10
print("b=", b)

a+b

## Notice, if a cell ends with a Python expression, the result
## of this expression is displayed when the code cell is run
```

```
Value of a: 1
b= 11
```

Out[1]:
```
12
```

In [2]:
```
### however, only the value of the last expression will be rendered

a
b
a+b
```

Out[2]:
```
12
```

In [3]:
```
### notice one other VERY IMPORTANT THING
### the values of the variables defined in one code block
### persist into the next

print(a, b)
```

```
1 11
```

In [4]:
```
print(a,b)
```

```
1 11
```

In [5]:
```
### While we won't use it much, you can
### use input() function inside a cell to read an input value
### by default, input values are strings, so if you want numbers
### you need to convert input values explicitly.
n = input("Enter a value")

print(n, type(n))

x = int(n)

print(x*x)
```

```
Enter a value54
54 <class 'str'>
2916
```

Task 2. Create a code cell below that uses a variable `myname`, assigns it the string containing your first and last name, and then prints out the phrase

> My name is [myname]

(replace `[myname]` with the value you assigned to it)

In [6]:
```
myname = "Maxim Veremeev"
print(f"My name is {myname}.")
```

```
My name is Maxim Veremeev.
```

Why use notebooks instead of scripts?

Advantage 1

Notebooks integrate code, text, and output in a single document. Since data science is not only about writing code, but also about interpreting and communicating the results, it is useful to have everything in one place.

Throughout the course, when working on your assignments, you will be **both** writing code in the code cells of the notebooks, **and** writing textual descriptions, explanations, and interpretations of your results in the text cells. This is standard practice in quite a lot of Data Science work.

The artifacts generated this way - with text, code, and code output, allow data scientists to tell rich stories with data.

Task 3. Take a look at the cell below. It contains a simple Python function (what does it do?) and tests.

Run the cell first, observe the output. As discussed above, one of the calls to the function (we can call it *one of the test cases*) does not produce output.

Create additional code cells below the cell that contains the definition of the `convert_celsius_to_fahrenheit` function, and move our test cases there. Make sure that the output of each function call can be displayed. You can add `print()` statements, in you want to.

Add another code cell, and add another call to the `convert_celsius_to_fahrenheit` function - try to come up with an unusual temperature value to convert.

In [7]:
```
### convert_celsius_to_fahrenheit()
### intput parameter: temperature_in_celsius – numeric value representing te
### returns: temperature in degrees Fahrenheit

def convert_celsius_to_fahrenheit(temperature_in_celsius):
  return 9 / 5 * temperature_in_celsius + 32
```

In [8]:
```
print(convert_celsius_to_fahrenheit(0))

print(convert_celsius_to_fahrenheit(100))
```

```
32.0
212.0
```

In [9]:
```
print(convert_celsius_to_fahrenheit(-745))
```

```
-1309.0
```

Task 4 (complete this task after the class)

add two or more (as needed) code cells below. In the first code cell you add, define a function `f2c`, which does the **inverse** of what `convert_celsius_to_fahrenheit` computes.

In the code cells after, create some test cases that

1. run the `f2c` function on some inputs
2. **chain** the function calls to `f2c` and `convert_celsius_to_fahrenheit`.
3. What should be the effect of chaining these functions (i.e., calling one on the output produced by the other on *some* input value)? Can you observe this behavior. After your code cells, add a text cell describing what you have observed and discussing it.

In [10]:
```
def f2c(temperature_in_fahrenheit):
  return (temperature_in_fahrenheit - 32) * (5 / 9)
```

In [11]:
```
a = f2c(50)
print(a)

b = f2c(0)
print(b)
```

```
10.0
-17.77777777777778
```

In [12]:
```
print(convert_celsius_to_fahrenheit(a))
print(convert_celsius_to_fahrenheit(b))
```

```
50.0
0.0
```

Using `f2c` on a value and then `using convert_celsius_to_fahrenheit` on that output converts a value to celsius from fahrenheit, and then from fahrenheit to celsius, resulting in the same output as the input.

Advantage 2

Notebooks allow for a more interactive style of programming. You don't have to write your entire program before executing it!

You can build up code gradually. You can do a computation in one cell and follow up in another cell.

In the example below, the function `convert_celsius_to_fahrenheit` was defined when we ran the code cell above. The notebook maintains a "memory" of all the variables and functions that have been defined.

In [13]:
```
convert_celsius_to_fahrenheit(0)
```

Out[13]: 32.0

A Caution about Notebooks

The code cells in a notebook are usually designed to be run in order, from top to bottom. Bad things can happen if you run them out of order.

Let's start from a clean state. Let's clear all the variables that we have defined. Go to Runtime > Restart runtime. What happens if you run the code cell below, without first running the cells above it?

In [14]:
```
convert_celsius_to_fahrenheit(20)
```

Out[14]:
```
68.0
```

Questions for Discussion

1. What do you think the numbers (e.g., `[1]`, `[2]`, `[3]`, ...) on the left of the code cells indicate?
2. Run the code cell below. Why doesn't the cell output anything? How would you modify the code so that it outputs the temperature in Fahrenheit?

In [15]:
```
temperature_in_fahrenheit = convert_celsius_to_fahrenheit(30)
```

It doesn't output anything because it is defining a variable, not computing a raw expression, which Colab would automatically show the output of, and doesn't have a `print` statement.

A Python Exercise

We expect you to have a basic familiarity with Python syntax and data structures. For example, you should know how to write `for` loops and work with lists and dicts.

Here is an exercise that should help you review your Python skills. The code below reads in data about Nobel laureates from the Internet into a Python dict called `nobel_laureates`. (You are not expected to understand this code.)

In [16]:
```
import requests
headers = {
    "User-Agent": "Mozilla/5.0"
}
response = requests.get("https://api.nobelprize.org/v1/laureate.json",header
nobel_laureates = response.json()
```

Here is your task: write a loop over the dict `nobel_laureates` to count the number of Nobel laureates that were affiliated with Stanford at the time of their prize.

If you want to see what this dict looks like, you can either:

1. open the URL in your browser, or
2. type `nobel_laureates` as the last line of a cell and run the cell.

In [17]:
```
nobel_laureates['laureates'][0]
```

Out[17]:
```
{'id': '1',
 'firstname': 'Wilhelm Conrad',
 'surname': 'Röntgen',
 'born': '1845-03-27',
 'died': '1923-02-10',
 'bornCountry': 'Prussia (now Germany)',
 'bornCountryCode': 'DE',
 'bornCity': 'Lennep (now Remscheid)',
 'diedCountry': 'Germany',
 'diedCountryCode': 'DE',
 'diedCity': 'Munich',
 'gender': 'male',
 'prizes': [{'year': '1901',
   'category': 'physics',
   'share': '1',
   'motivation': '"in recognition of the extraordinary services he has rend
ered by the discovery of the remarkable rays subsequently named after hi
m"',
   'affiliations': [{'name': 'Munich University',
     'city': 'Munich',
     'country': 'Germany'}]}]}
```

In [18]:
```
type (nobel_laureates['laureates'])
```

Out[18]:
```
list
```

In [19]:
```
# stanford_counter = 0;
# for laureate in nobel_laureates['laureates']:
#   if nobel_laureates['laureates']['affiliations']['name'] == "Stanford Uni
#     stanford_counter += 1;
# print('stanford_counter')
```

In [20]:
```
stanford_count = 0
other_count = 0

for data in nobel_laureates['laureates']:
    for prize in data['prizes']:
        for affiliation in prize['affiliations']:
            if not isinstance(affiliation, dict):
                continue
            if affiliation['name'] == "Stanford University":
                stanford_count += 1
            else:
                other_count += 1

print("Amount of Stanford Nobel laureates: ", stanford_count)
print("Amount of non-Stanford affiliations: ", other_count)
print("Total amount of affiliations: ", stanford_count + other_count)
```

```
Amount of Stanford Nobel laureates:  22
Amount of non-Stanford affiliations:  825
Total amount of affiliations:  847
```

(Optional) Running Notebooks on Your Own Computer

In this class, all of the notebooks will be hosted on Colab, so you do not have to install anything on your own machine.

But if you want to run notebooks locally (on your own machine), you should:

1. Install Anaconda.
2. Open Anaconda Navigator and launch either JupyterLab or Notebook.

Submission Instructions

- Restart this notebook and run the cells from beginning to end.
  - Go to Runtime > Restart Session and Run All.

In [ ]:
```
# @markdown Run this cell to download this notebook as a webpage, `_NOTEBOOK

import google, json, nbformat

# Get the current notebook and write it to _NOTEBOOK.ipynb
raw_notebook = google.colab._message.blocking_request("get_ipynb",
                                                      timeout_sec=30)["ipynb
with open("_NOTEBOOK.ipynb", "w", encoding="utf-8") as ipynb_file:
  ipynb_file.write(json.dumps(raw_notebook))

# Use nbconvert to convert .ipynb to .html.
!jupyter nbconvert --to html --log-level WARN _NOTEBOOK.ipynb
```

```
# Download the .html file.
google.colab.files.download("_NOTEBOOK.html")
```

- Open `_NOTEBOOK.html` in your browser, and save it as a PDF.
  - Go to File > Print > Save as PDF.
- Double check that all of your code and output is visible in the saved PDF.
- Upload the PDF to Gradescope.
  - Please be sure to select the correct pages corresponding to each question.